深度学习
第 3 章　神经网络的学习

——Loss、梯度、SGD：搞懂这三件事，神经网络就真的会”学”了

2026 年 3 月 17 日

本章题目导览：你将挑战哪些核心问题？

前两章我们搭出了神经网络的”骨架”——知道数据怎么流过网络、激活函数是什么。但网络还不会”学”，它不知道自己预测得好不好，也不知道怎么变得更好。本章解决这两个问题，共 6 道题：

- **Q3-01（辨析填空）**：常见损失函数横向对比——MSE、MAE、交叉熵、Huber Loss，各自的公式、适用任务和主要特点是什么？
- **Q3-02（计算题）**：损失函数手动计算——给定预测值和真实标签，分别计算 MSE 损失和交叉熵损失，把公式变成具体数字，感受”哪种惩罚更重”。
- **Q3-03（概念理解）**：导数、偏导数与梯度——三个概念的联系与区别，以及数值微分的原理和计算。
- **Q3-04（推导计算）**：神经网络的梯度计算——对一个简单的单层网络，手动用数值微分计算损失对权重的梯度，理解梯度的物理含义。
- **Q3-05（对比分析）**：梯度下降三兄弟——批量梯度下降（BGD）、随机梯度下降（SGD）、小批量梯度下降（Mini-batch GD）的区别、优缺点和适用场景。
- **Q3-06（综合代码实现）**：训练循环全流程——阅读并补全一个含前向传播、损失计算、梯度更新的完整训练代码，追踪每一步的数据变化。

1　第 3 章　神经网络的学习

前两章搭好了网络的骨架，但它还不会”学”。给它一张猫的图片，它输出一个随机的预测值，反正也不知道对不对，更不知道怎么改进。

要让网络”学起来”，需要解决三件事：

- **怎么衡量”预测有多差”？**——损失函数（Loss Function）
- **怎么知道”往哪个方向调整参数”？**——梯度（Gradient）
- **怎么一步步调整让 Loss 变小？**——梯度下降（Gradient Descent）

这三件事合起来，就是神经网络”学习”的完整闭环。本章把这个闭环从数学到代码全部打通。

先建立一个总感觉：学习是什么过程？

把神经网络的学习比作一个蒙眼射箭手在雾中找靶心。

- **损失函数** = 报靶员：射完一箭后，告诉你”偏了多少米”——这就是 Loss，Loss 越小越好；
- **梯度** = 指南针：告诉你”要让 Loss 减小，弓应该往哪个方向转”——梯度指向 Loss 增大最快的方向，所以朝梯度的反方向调整；
- **梯度下降** = 每次微调弓的角度：根据指南针，往减小 Loss 的方向小步移动，一轮一轮地迭代，越来越接近靶心；
- **SGD** = 每次不用把所有箭都射一遍再调整，而是射一箭（或一小批箭）就调整一次——虽然每次估计可能有误差，但调整频率更高，总体反而更快。

Q3-01 常见损失函数横向对比

题目描述：

文档第 3.1 节介绍了多种损失函数。损失函数是”评分系统”——选对了，模型才能朝正确方向优化；选错了，优化方向就会偏离目标。

问题：

(1) 填写下表，梳理四种常见损失函数的关键信息：

损失函数	数学公式	适用任务	主要特点
均方误差（MSE）	$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$	_____	_____
平均绝对误差（MAE）	$\mathcal{L} = \frac{1}{N}\sum_{i=1}^{N}\lvert\hat{y}_i - y_i\rvert$	_____	_____
交叉熵（CE）	$\mathcal{L} = -\frac{1}{N}\sum_{i=1}^{N}\sum_{k=1}^{K} t_{ik}\log\hat{y}_{ik}$	_____	_____
Huber Loss	$\mathcal{L}_\delta = \begin{cases} \frac{1}{2}(\hat{y}-y)^2 & \lvert\hat{y}-y\rvert \le \delta \\ \delta\lvert\hat{y}-y\rvert - \frac{\delta^2}{2} & \text{otherwise} \end{cases}$	_____	_____

(2) MSE 和 MAE 都用于回归任务，但对**离群点（outliers）**的敏感程度不同。解释为什么 MSE 比 MAE 对离群点更敏感，以及各自适合什么数据分布。

(3) 为什么分类任务通常使用交叉熵损失而不是 MSE？从梯度角度解释：当预测结果很差时（预测概率接近 0 而真实标签为 1），两种损失函数的梯度有何差异？

(4) Huber Loss 被称为”MSE 和 MAE 的结合体”。解释为什么，以及超参数 δ 如何控制这种平衡。

来源溯源：[文档第 3 章 3.1 损失函数]

小白必读：损失函数为什么不能随便选？

- **损失函数决定了优化目标**：梯度下降会让 Loss 减小——你选什么 Loss，模型就朝那个方向优化。选 MSE，模型会努力让平均平方误差最小；选交叉熵，模型会努力让预测概率分布和真实分布接近。目标不同，训练出来的模型行为就不同。
- **分类为什么不用 MSE**：分类标签是 $\{0,1\}$ 或 one-hot 向量，MSE 在两端（预测值接近 0 或 1）梯度极小，训练极慢；而交叉熵在预测错误时（预测概率接近 0 但真实为 1）梯度很大，能快速纠错。
- **损失函数要可微**：梯度下降依赖梯度，损失函数在大多数点必须可微。MAE 在零点不可微（但在实践中可用次梯度或 Huber 替代），准确率（Accuracy）完全不可微，因此不能直接作为损失函数，只能作为评估指标。

参考答案与详细解析

(1) 四种损失函数对比表：

损失函数	公式	适用任务	主要特点
MSE	$\frac{1}{N}\sum(\hat{y}_i - y_i)^2$	回归	平滑可微，对离群点敏感
MAE	$\frac{1}{N}\sum\lvert\hat{y}_i - y_i\rvert$	回归	对离群点鲁棒，零点不可微
交叉熵	$-\frac{1}{N}\sum_i\sum_k t_{ik}\log\hat{y}_{ik}$	分类	配合 Softmax，梯度信号强
Huber	小误差用 MSE，大误差用 MAE	回归	两者折中，δ 控制边界

(2) MSE vs MAE 对离群点的敏感度：

MSE 对误差**平方**，一个误差为 10 的离群点贡献 $10^2 = 100$，而误差为 1 的正常点贡献 1。离群点对 MSE 的影响是正常点的 100 倍。MAE 对误差取绝对值，离群点贡献 10，正常点贡献 1，差距仅 10 倍。

- **数据无离群点/误差服从正态分布**：选 MSE（最大似然估计在正态噪声假设下等价于 MSE）；
- **数据含离群点/误差服从重尾分布**：选 MAE 或 Huber Loss。

(3) 交叉熵 vs MSE 用于分类——梯度角度：

设真实标签 $y = 1$，预测概率 $\hat{y} \approx 0$（预测严重错误）。

交叉熵梯度：$\partial\mathcal{L}/\partial\hat{y} = -1/\hat{y}$，当 $\hat{y} \to 0$ 时梯度趋向无穷大——**预测越差，纠错信号越强**，收敛快。

MSE 梯度：$\partial\mathcal{L}/\partial\hat{y} = 2(\hat{y} - y) \approx -2$，无论预测多差，梯度幅度几乎不变，纠错信号微弱，训练极慢。

此外，Sigmoid/Softmax 输出层与 MSE 组合后，梯度中会出现 $\sigma'(z)$ 项（最大值仅 0.25），进一步削弱梯度。

(4) Huber Loss 的双重特性：

- 误差较小时（$\lvert\hat{y} - y\rvert \le \delta$）：退化为 MSE（$\frac{1}{2}e^2$），平滑可微，梯度稳定；
- 误差较大时（超过 δ）：退化为 MAE（线性惩罚），不再随误差平方增长，抑制离群点影响。

δ 控制”什么程度算大误差”：δ 越大，越接近纯 MSE；δ 越小，越接近纯 MAE。实践中 δ 通常通过交叉验证选取。

Q3-02 损失函数手动计算：回归 vs 分类

题目描述：

文档第 3.1.2 和 3.1.3 节分别介绍了分类和回归的损失函数。把公式变成具体数字，是真正理解损失函数的关键一步。

场景一：回归任务（房价预测）

4 个样本的真实房价（万元）为 $y = [100, 200, 150, 300]$，模型预测值为 $\hat{y} = [110, 185, 160, 280]$。

场景二：分类任务（三分类）

单个样本，真实类别为第 2 类（one-hot 标签 $t = [0, 1, 0]$），模型经过 Softmax 后的预测概率为 $\hat{y} = [0.1, 0.7, 0.2]$。

问题：

(1) 对场景一，计算 **MSE 损失**（结果保留两位小数）：

$$\mathcal{L}_{\text{MSE}} = \frac{1}{N}\sum_{i=1}^{N}(\hat{y}_i - y_i)^2$$

(2) 对场景一，计算 **MAE 损失**（结果保留两位小数）：

$$\mathcal{L}_{\text{MAE}} = \frac{1}{N}\sum_{i=1}^{N}|\hat{y}_i - y_i|$$

(3) 对场景二，计算**交叉熵损失**（结果保留三位小数）：

$$\mathcal{L}_{\text{CE}} = -\sum_{k=1}^{K} t_k \log \hat{y}_k$$

(4) 若场景二中预测完全正确（$\hat{y} = [0, 1, 0]$），交叉熵损失理论上应为多少？实际上为什么不能真的输入 $\hat{y}_k = 0$ 或 $\hat{y}_k = 1$（提示：$\log 0 = -\infty$）？PyTorch 的 `nn.CrossEntropyLoss` 如何解决这个问题？

(5) 比较 (1) 和 (2) 的结果：对于同一组预测误差，MSE 和 MAE 的数值大小有什么规律？MSE 比 MAE 更能”放大”哪种误差？

来源溯源：[文档第 3 章 3.1.2 分类任务损失函数 / 3.1.3 回归任务损失函数]

小白必读：交叉熵的直觉——它在衡量什么？

- **交叉熵衡量"概率分布的差距"**：真实标签 t 是一个概率分布（one-hot 向量），预测值 $\hat{y}$ 是另一个概率分布（Softmax 输出）。交叉熵越小，两个分布越接近。
- **为什么只有正确类别的项起作用**：one-hot 标签中，错误类别的 $t_k = 0$，对应项 $t_k \log \hat{y}_k = 0$，消失了。所以实际上交叉熵就是 $-\log$(正确类别的预测概率)——预测正确类别的概率越高，Loss 越小，这非常符合直觉。
- **完美预测时 Loss = 0**：若 $\hat{y}_k = 1$（正确类别），$-\log(1) = 0$，Loss 为零——但 Softmax 的输出永远不会真的等于 0 或 1（只能无限接近），所以实践中 Loss 永远大于 0。

参考答案与详细解析

(1) MSE 损失计算：

各样本误差：10，-15，10，-20；误差平方：100，225，100，400。

$$\mathcal{L}_{\text{MSE}} = \frac{100 + 225 + 100 + 400}{4} = \frac{825}{4} = \boxed{206.25}\,(\text{万元})^2$$

(2) MAE 损失计算：

各样本绝对误差：10，15，10，20。

$$\mathcal{L}_{\text{MAE}} = \frac{10 + 15 + 10 + 20}{4} = \frac{55}{4} = \boxed{13.75}\ \text{万元}$$

(3) 交叉熵损失计算：

真实标签 $t = [0, 1, 0]$，预测概率 $\hat{y} = [0.1, 0.7, 0.2]$。只有第 2 类（$k = 2$）的 $t_k = 1$ 起作用：

$$\mathcal{L}_{\text{CE}} = -(0 \cdot \log 0.1 + 1 \cdot \log 0.7 + 0 \cdot \log 0.2) = -\log(0.7) \approx \boxed{0.357}$$

(4) 数值稳定性处理：

理论上完美预测（$\hat{y}_2 = 1$）时 $\mathcal{L} = -\log(1) = 0$。

但实际中 $\hat{y}_k = 0$ 会导致 $\log(0) = -\infty$，产生 `NaN`，数值计算崩溃。

PyTorch 的解决方案：`nn.CrossEntropyLoss` 在内部将 Softmax 和 CrossEntropy 合并，使用 **log-sum-exp** 技巧在对数域直接计算，同时加入数值稳定性修正（减去最大 logit），避免 e^{z_k} 溢出，也避免 $\log 0$ 问题。这也是为什么该函数要求输入 logits 而非概率。

(5) MSE vs MAE 数值对比与规律：

同一组预测误差，MSE = 206.25，MAE = 13.75，**MSE 远大于 MAE**（单位也不同：MSE 的单位是万元的平方）。

MSE 特别放大**大误差**：第 4 个样本误差为 20，在 MAE 中贡献 20，在 MSE 中贡献 400（是其他误差为 10 的样本的 4 倍）。离群点对 MSE 的影响呈平方增长，这既是 MSE 的"缺点"（离群点主导训练），也是优点（对大误差施加更强的惩罚，迫使模型优先改善大误差）。

Q3-03 导数、偏导数与梯度：三个概念辨析

题目描述：

文档第 3.2 节介绍了数值微分的基本概念。梯度是深度学习中最核心的数学工具，但”导数”“偏导数”“梯度”三个词经常被混用。本题把它们彻底分清楚。

问题：

(1) **概念辨析**：填写下表，区分三个概念：

概念	适用对象	含义
导数 $f'(x)$	单变量函数 $f(x)$	_____
偏导数 $\partial f/\partial x_i$	多变量函数 $f(x_1,\ldots,x_n)$	_____
梯度 ∇f	多变量函数 $f(x_1,\ldots,x_n)$	_____

(2) **数值微分计算**：数值微分用差商近似导数。中心差分公式为：

$$f'(x) \approx \frac{f(x+h)-f(x-h)}{2h}$$

对函数 $f(x)=x^3-2x$，取 $h=10^{-4}$，用数值微分估计 $f'(2)$（结果保留四位小数），并与解析导数 $f'(x)=3x^2-2$ 的精确值比较误差。

(3) **梯度计算**：对二元函数 $f(x_1,x_2)=x_1^2+2x_1x_2+3x_2^2$，求梯度 ∇f 的解析表达式，并计算在点 $(1,2)$ 处的梯度向量。

(4) **梯度的几何含义**：梯度向量的方向指向函数值增大最快的方向，其大小（模）表示该方向上的变化率。梯度下降沿梯度的**负方向**更新参数，解释为什么是”负方向”，以及为什么步长（学习率）不能太大或太小。

来源溯源：[文档第 3 章 3.2 数值微分]

小白必读：梯度是什么？用最通俗的语言

- **导数**（单变量）：函数在某点的”斜率”——往右走一小步，函数值涨了多少？
- **偏导数**（多变量，固定其他变量）：在多维空间中，只沿着某一个坐标轴方向走一小步，函数值涨了多少？
- **梯度**（多变量，所有偏导数打包）：把每个方向的偏导数打包成一个向量 $[\partial f/\partial x_1, \partial f/\partial x_2, \ldots]$。这个向量指向”函数值上升最快的方向”。
- **直觉类比**：你站在一座山上，梯度就是”最陡上坡方向”——梯度下降就是沿着最陡下坡方向走，找山谷（最低点/最小 Loss）。

生活比喻：梯度就是地形图上的等高线垂线

想象一张地形图，等高线越密说明坡度越陡。

- **偏导数** $\partial f/\partial x_1$：只沿东西方向走，看海拔怎么变——等高线在东西方向的变化率；
- **偏导数** $\partial f/\partial x_2$：只沿南北方向走，看海拔怎么变；
- **梯度向量**：把东西变化率和南北变化率合成一个箭头，这个箭头精确指向" 最陡上坡方向"；
- **梯度下降**：沿着这个箭头的反方向走，朝山谷滑去。步子太大（学习率过大）会越过山谷、反弹；步子太小（学习率过小）爬到山谷要走很久。

参考答案与详细解析

(1) 概念辨析：

概念	适用对象	含义
导数 $f'(x)$	单变量函数	x 变化一点时，$f(x)$ 变化多快；即函数在该点的切线斜率
偏导数 $\partial f/\partial x_i$	多变量函数	固定其他变量，只让 x_i 变化时，f 的变化率
梯度 ∇f	多变量函数	所有偏导数组成的向量，指向函数值增大最快的方向

(2) 数值微分计算（$f(x) = x^3 - 2x$，$x = 2$）：

$$f(2+h) = (2.0001)^3 - 2(2.0001) \approx 8.000600 - 4.0002 = 4.0004$$

$$f(2-h) = (1.9999)^3 - 2(1.9999) \approx 7.999400 - 3.9998 = 3.9996$$

$$f'(2) \approx \frac{4.0004 - 3.9996}{2 \times 10^{-4}} = \frac{0.0008}{0.0002} = \boxed{10.0000}$$

解析导数：$f'(x) = 3x^2 - 2$，$f'(2) = 12 - 2 = 10$。数值微分结果与解析值完全一致（中心差分的误差为 $O(h^2)$，$h = 10^{-4}$ 时误差约 10^{-8}，在四位小数内无法体现）。

(3) 梯度计算（$f(x_1, x_2) = x_1^2 + 2x_1x_2 + 3x_2^2$）：

$$\frac{\partial f}{\partial x_1} = 2x_1 + 2x_2, \quad \frac{\partial f}{\partial x_2} = 2x_1 + 6x_2$$

$$\nabla f = \begin{pmatrix} 2x_1 + 2x_2 \\ 2x_1 + 6x_2 \end{pmatrix}$$

在点 $(1, 2)$ 处：

$$\nabla f(1,2) = \begin{pmatrix} 2 \times 1 + 2 \times 2 \\ 2 \times 1 + 6 \times 2 \end{pmatrix} = \boxed{\begin{pmatrix} 6 \\ 14 \end{pmatrix}}$$

(4) 梯度负方向与学习率的原因：

梯度指向 f **增大**最快的方向。我们的目标是让 Loss **减小**，所以必须朝梯度的**负方向**更新参数：$w \leftarrow w - \eta \nabla f$。

- **学习率 η 过大**：每步迈得太大，会跨过 Loss 的最低点，在两侧反复震荡，可能发散（Loss 越来越大）；
- **学习率 η 过小**：步子太小，收敛极慢，需要很多轮才能到达最优解，训练时间过长；
- **最优学习率**：使每步更新都朝最优解迈进而不震荡，通常通过实验或学习率衰减策略找到。

Q3-04 神经网络的梯度计算

题目描述：

文档第 3.3 节介绍了如何计算神经网络的梯度。对于一个单层神经网络（线性层 + Sigmoid + 二分类交叉熵损失），我们来手动用数值微分估算梯度。

网络结构：$z = w \cdot x + b$，$\hat{y} = \sigma(z)$，$L = -[y \log \hat{y} + (1-y)\log(1-\hat{y})]$

当前参数：$w = 0.5$，$b = 0$，$x = 2.0$，$y = 1$（真实标签），$h = 10^{-4}$。

问题：

(1) **前向传播基准值**：计算当前参数下的 z、$\hat{y}$、L（结果保留四位小数）。

(2) **数值微分估算** $\partial L/\partial w$：

a. 将 w 增加 h（即 $w_+ = 0.5 + 10^{-4}$），计算 $L(w_+)$；

b. 将 w 减少 h（即 $w_- = 0.5 - 10^{-4}$），计算 $L(w_-)$；

c. 用中心差分 $\partial L/\partial w \approx (L(w_+) - L(w_-))/(2h)$ 估算梯度（结果保留四位小数）。

(3) **数值微分估算** $\partial L/\partial b$：同理对 b 做数值微分，估算 $\partial L/\partial b$。

(4) **梯度的物理含义**：根据计算结果，$\partial L/\partial w$ 的正负说明了什么？如果使用学习率 $\eta = 0.1$，执行一步梯度下降后，新的 w 和 b 分别是多少？

(5) **解析梯度验证**：对于 Sigmoid + 二分类交叉熵的组合，可以推导出：

$$\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot x, \quad \frac{\partial L}{\partial b} = \hat{y} - y$$

用上述公式计算解析梯度，与数值微分结果对比，误差是多少？

来源溯源：[文档第 3 章 3.3 神经网络的梯度计算]

小白必读：为什么要”手算”梯度这件事？

- **建立直觉**：手算一遍梯度，你才能真正理解”梯度告诉你什么”——它的正负说明参数应该增大还是减小，它的大小说明调整幅度应该多大。
- **理解数值微分的用途**：数值微分（有限差分）是验证解析梯度是否正确的”金标准”——虽然计算量大，但不需要推导公式，只要能前向计算损失就行。
- **感受链式法则的威力**：从本题可以看出，即使是最简单的单节点网络，数值微分也需要多次前向传播。真实网络有数百万参数，解析梯度（反向传播）一次性算出所有梯度，效率高出数量级。

参考答案与详细解析

(1) 前向传播基准值（$w = 0.5$，$b = 0$，$x = 2$，$y = 1$）：

$$z = 0.5 \times 2 + 0 = 1.0$$

$$\hat{y} = \sigma(1.0) = \frac{1}{1 + e^{-1}} = \frac{1}{1 + 0.3679} \approx 0.7311$$

$$L = -\log(0.7311) \approx \boxed{0.3133}$$

(2) 数值微分估算 $\partial L/\partial w$：

(a) $w_+ = 0.5001$：$z_+ = 0.5001 \times 2 = 1.0002$，$\hat{y}_+ = \sigma(1.0002) \approx 0.7312$，$L_+ = -\log(0.7312) \approx 0.3132$

(b) $w_- = 0.4999$：$z_- = 0.4999 \times 2 = 0.9998$，$\hat{y}_- = \sigma(0.9998) \approx 0.7310$，$L_- = -\log(0.7310) \approx 0.3134$

(c) 中心差分：

$$\frac{\partial L}{\partial w} \approx \frac{0.3132 - 0.3134}{2 \times 10^{-4}} = \frac{-0.0002}{0.0002} = \boxed{-0.5378}$$

（精确数值视舍入而定，约为 -0.5378）

(3) 数值微分估算 $\partial L/\partial b$：

$b_\pm = \pm 10^{-4}$，$z_\pm = 1.0 \pm 10^{-4}$，差值引起的 L 变化为 $\hat{y} - 1$ 的量级：

$$\frac{\partial L}{\partial b} \approx \boxed{-0.2689}$$

(4) 梯度物理含义与一步更新：

$\partial L/\partial w \approx -0.5378 < 0$：表示增大 w 会使 Loss 减小，即 w 目前偏小，应增大。梯度下降沿负梯度方向（即正方向）更新 w。

一步梯度下降（$\eta = 0.1$）：

$$w_{\text{new}} = 0.5 - 0.1 \times (-0.5378) = 0.5 + 0.0538 = \boxed{0.5538}$$

$$b_{\text{new}} = 0 - 0.1 \times (-0.2689) = \boxed{0.0269}$$

(5) 解析梯度验证：

$$\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot x = (0.7311 - 1) \times 2 = -0.2689 \times 2 = \boxed{-0.5378}$$

$$\frac{\partial L}{\partial b} = \hat{y} - y = 0.7311 - 1 = \boxed{-0.2689}$$

解析梯度与数值微分结果完全吻合（误差在 10^{-8} 量级，远小于四位小数精度），验证正确。✓

关键结论：Sigmoid + 二元交叉熵的梯度极简公式

$$\frac{\partial L}{\partial w} = (\hat{y} - y) \cdot x, \quad \frac{\partial L}{\partial b} = \hat{y} - y$$

含义：预测概率 $\hat{y}$ 和真实标签 y 之差就是误差信号，直接乘以对应输入 x 即为权重梯度。预测越准（$\hat{y} \to y$），梯度越小，更新越慢——这正是我们希望的行为。

Q3-05 梯度下降三兄弟：BGD、SGD、Mini-batch GD

题目描述：

文档第 3.4 节介绍了随机梯度下降法。”梯度下降”并不是一种固定的算法，根据每次更新时使用的数据量，分为三种变体，各有优缺点。

问题：

(1) 填写下表，对比三种梯度下降变体：

方法	每次更新使用的数据量	主要优点	主要缺点
批量梯度下降（BGD）	全部训练集 N 个样本	_____	_____
随机梯度下降（SGD）	单个样本	_____	_____
小批量梯度下降（Mini-batch）	B 个样本（$1 < B < N$）	_____	_____

(2) **Epoch 与 Iteration 的区别**：设训练集有 $N = 10000$ 个样本，batch size $B = 100$，训练 20 个 epoch。计算：

a. 每个 epoch 包含多少次参数更新（iterations）?

b. 总共进行多少次参数更新?

c. 总共处理了多少个样本次数（样本 × 次数）?

(3) **为什么 SGD 有时比 BGD 收敛更快**：从”逃离局部极小值”和”更新频率”两个角度解释。

(4) **Batch Size 的选择**：以下三种说法中，哪些是正确的？请逐一判断并说明理由：

a. ”Batch size 越大，模型泛化性能越好”

b. ”Batch size 过小，GPU 利用率低，训练慢”

c. ”Batch size 的选择和学习率应该一起调整”

(5) **数据打乱（Shuffle）**：为什么每个 epoch 开始前通常要对训练数据进行 shuffle（随机打乱）？如果不打乱，会出现什么问题？

来源溯源：[文档第 3 章 3.4 随机梯度下降法（SGD）]

生活比喻：三种梯度下降像三种市场调研方式

你是一家公司的产品经理，想知道产品需要改进哪里（梯度方向）：

- **BGD（全量调研）**：把所有 100 万用户都调查一遍，再做决策。结果最准确，但等完所有反馈才能改一次，太慢了；
- **SGD（随机抽一人）**：随机问一个用户的意见，立刻就改。改得飞快，但这个用户可能是异类，反馈噪声很大，产品走走停停；
- **Mini-batch（抽一小组）**：每次随机抽 100 人调研，综合他们的意见再改。既比全量快得多，反馈又比单人靠谱——实践中几乎所有深度学习都用这种方式。

参考答案与详细解析

(1) 三种梯度下降对比表：

方法	数据量	主要优点	主要缺点
BGD	全部 N 个	梯度估计准确，收敛稳定	内存需求大，每步耗时长
SGD	1 个	更新频率极高，可逃离局部极小	梯度噪声大，Loss 震荡，难并行
Mini-batch	B 个	平衡速度与准确性，可充分利用 GPU 并行	需调 batch size 超参数

(2) Epoch 与 Iteration 计算：

a. 每个 epoch 的 iterations $= N/B = 10000/100 = \boxed{100}$ 次

b. 总参数更新次数 $= 100 \times 20 = \boxed{2000}$ 次

c. 总处理样本次数 $= 10000 \times 20 = \boxed{200000}$ 个样本次数

(3) SGD 有时比 BGD 收敛更快的原因：

- **逃离局部极小值**：BGD 每次沿精确梯度更新，容易陷在局部极小值（停止更新）。SGD 的梯度噪声相当于给参数更新加了”随机扰动”，有机会跳出局部极小值，找到更好的解；
- **更新频率更高**：BGD 处理完全部 N 个样本才更新一次，SGD 处理 1 个样本就更新一次。在相同的数据处理量下，SGD 更新了 N 次，参数已经朝最优解迈进了 N 步，而 BGD 只走了 1 步。

(4) Batch Size 三种说法的判断：

a. **错误**。研究表明 batch size 越大，模型往往泛化性能**越差**（倾向于收敛到”尖锐的”极小值，泛化能力弱）。小 batch size 引入的噪声有助于找到”平坦的”极小值，泛化更好；

b. **正确**。GPU 的并行计算需要足够的数据量才能充分利用显存带宽。batch size $= 1$ 时，GPU 大部分时间在等待数据，利用率极低；

c. **正确**。增大 batch size 相当于减小了梯度的方差，等效于”学习率变小”。实践中有一个经验规律：**线性缩放规则**——batch size 增大 k 倍时，学习率也应增大 k 倍，以保持等效的参数更新幅度。

(5) Shuffle 的必要性：

如果数据按类别排列（如前 1000 张猫、后 1000 张狗），不打乱时每个 mini-batch 只包含单一类别，梯度估计严重偏置——模型只对当前批次的类别优化，损害了泛化性。此外，若数据有时间或顺序上的规律，不打乱会让模型学到顺序特征而非真实规律。

每个 epoch 前 shuffle 能保证每个 mini-batch 是训练集的随机子集，梯度估计更接近全数据集的真实梯度，训练更稳定，泛化更好。

Q3-06 综合代码实现：完整训练循环

题目描述：

文档第 3.5 节给出了神经网络学习的综合代码实现。以下是一个用 NumPy 实现的完整单层神经网络训练代码骨架（二分类任务），请阅读并补全关键步骤。

```python
import numpy as np

# ===== 辅助函数 =====
def sigmoid(z):
    return 1 / (1 + np.exp(-z))

def binary_cross_entropy(y_hat, y):
    eps = 1e-8  # 防止 log(0)
    return -np.mean(y * np.log(y_hat + eps)
                    + (1 - y) * np.log(1 - y_hat + eps))

def numerical_gradient(loss_fn, param, h=1e-4):
    grad = np.zeros_like(param)
    for i in range(param.size):
        param.flat[i] += h
        lp = loss_fn()
        param.flat[i] -= 2 * h
        lm = loss_fn()
        grad.flat[i] = (lp - lm) / (2 * h)
        param.flat[i] += h
    return grad

# ===== 数据与初始化 =====
np.random.seed(0)
X = np.random.randn(100, 3)    # 100 个样本，3 个特征
y = (X[:, 0] + X[:, 1] > 0).astype(float)  # 真实标签

W = np.random.randn(3, 1) * 0.01  # 权重
b = np.zeros(1)                   # 偏置
lr = 0.1

# ===== 训练循环 =====
for epoch in range(50):
    # ------ 步骤A：前向传播 ------
    Z     = ______________          # 线性变换：[100,3] @ [3,1] + [1]
    Y_hat = ______________          # Sigmoid 激活

    # ------ 步骤B：计算损失 ------
    loss = ______________

    # ------ 步骤C：计算梯度（用解析梯度）------
    dZ = ______________             # Sigmoid+CE 组合梯度：Y_hat - y（注意形状）
    dW =                            # dL/dW
```

小白必读：训练循环的四步节奏，永远不会变

- **第一步，前向传播**：数据流过网络，计算预测值；
- **第二步，计算 Loss**：预测值和真实标签比较，得到一个数字；
- **第三步，计算梯度**：Loss 对每个参数的偏导数（方向盘告诉你往哪转）；
- **第四步，更新参数**：沿梯度负方向走一步（$w \leftarrow w - \eta \cdot \nabla_w L$）。

这四步就是深度学习训练的最小闭环。无论 PyTorch 的训练循环写得多复杂，拆开来看都是这四步：`forward` → `loss` → `backward` → `optimizer.step()`。

参考答案与详细解析

(1) 完整填写内容：

```python
# 步骤A：前向传播
Z     = X @ W + b                          # [100,3]@[3,1]+[1] = [100,1]
Y_hat = sigmoid(Z)                         # [100,1]

# 步骤B：计算损失
loss  = binary_cross_entropy(Y_hat, y.reshape(-1, 1))

# 步骤C：计算解析梯度
N  = X.shape[0]                            # 样本数 100
dZ = (Y_hat - y.reshape(-1, 1)) / N       # [100,1]，除以N取均值
dW = X.T @ dZ                             # [3,100]@[100,1] = [3,1]
db = np.sum(dZ, axis=0)                   # [1]，对样本求和

# 步骤D：参数更新
W -= lr * dW
b -= lr * db

```

(2) 形状追踪：

变量	形状	说明
X	[100, 3]	100 个样本，3 个特征
W	[3, 1]	3 个输入特征，1 个输出
b	[1]	标量偏置
Z	[100, 1]	每个样本的线性输出
Y_hat	[100, 1]	Sigmoid 后的预测概率
dZ	[100, 1]	每个样本的误差信号（除以 N）
dW	[3, 1]	与 W 形状相同
db	[1]	与 b 形状相同

(3) $\mathrm{d}W = X^T \cdot \mathrm{d}Z/N$ 的推导依据：

来自 Q3-04 中的单样本结论 $\partial L/\partial w = (\hat{y} - y) \cdot x$ 推广到批量情形，以及 Q4-05 中的 Affine 层矩阵梯度推导 $\partial L/\partial W = X^T \delta_Y$。

除以 N 是因为 `binary_cross_entropy` 对 N 个样本取了均值（`np.mean`），对应地，梯度也要对 N 个样本取均值——即 $\mathrm{d}Z$ 中已除以 N，$\mathrm{d}W = X^T \cdot \mathrm{d}Z$ 自然也是均值梯度。这保证了损失尺度和梯度尺度在不同 batch size 下保持一致。

(4) 训练过程观察：

正常情况下，Loss 应**单调下降**（可能有小幅波动），最终趋于平稳。

Loss 不降反升的最常见原因：

- **学习率过大**：每步更新幅度太大，越过最优点，Loss 震荡或发散——最常见原因，首先尝试将 η 减小 10 倍；
- **梯度计算错误**：符号写反（用了 $+\eta$ 而不是 $-\eta$），或梯度公式有误；
- **数值溢出**：e^z 在 z 很大时溢出为 `inf`，导致 Loss 变成 `NaN`。

(5) 解析梯度 vs 数值微分的效率：

步骤 C 使用的是**解析梯度**（直接用公式计算，一次前向传播后计算即完成）。

若改用 `numerical_gradient`：本例共有 $3 \times 1 + 1 = 4$ 个参数，每个参数需要 2 次前向传播，共需 $4 \times 2 = 8$ 次前向传播才能算出一次梯度。对真实网络（如参数量 10^7），数值微分需要 2×10^7 次前向传播，而解析梯度（反向传播）只需 1 次，**效率差距约** 2×10^7 **倍**——数值微分在实际训练中完全不可行。

完整训练循环模板（可直接复用）

```python
for epoch in range(num_epochs):
    # 1. 前向传播
    Z     = X @ W + b
    Y_hat = sigmoid(Z)

    # 2. 计算损失
    loss = binary_cross_entropy(Y_hat, y)

    # 3. 计算梯度（解析梯度）
    dZ = (Y_hat - y) / N
    dW = X.T @ dZ
    db = np.sum(dZ, axis=0)

    # 4. 更新参数
    W -= lr * dW
    b -= lr * db

```

现实应用场景

Mini-batch SGD——支撑起整个深度学习时代的训练策略：
本章看似”简单”的随机梯度下降，是所有现代大模型训练的基础。GPT 系列在预训练阶段，使用 batch size 数百万 token、学习率精心调节的 AdamW（SGD 的改进版），在数千个 GPU 上同时运行 mini-batch 梯度下降。理解”每个 batch 算一次梯度、更新一次参数”这个核心循环，就理解了深度学习训练的本质——无论规模大到何种程度，这个闭环从未改变。

2 本章小结：学习的三件事全打通了

学完这 6 道题，整章内容可以浓缩成这一句话：

$$\textbf{用损失函数量化错误} \xrightarrow{\textbf{梯度}} \textbf{知道往哪改} \xrightarrow{\textbf{SGD}} \textbf{一步步变好}$$

- **损失函数**：回归用 MSE/MAE/Huber，分类用交叉熵；选错方向，优化就跑偏；
- **梯度**：导数的多变量推广，方向指向 Loss 增大最快处；沿负梯度更新参数，Loss 就会减小；
- **数值微分**：用差商近似导数，是验证梯度的金标准，但计算量与参数数量成正比，实际训练不可用；
- **SGD 三兄弟**：BGD 准但慢，SGD 快但噪声大，Mini-batch 折中两者，是深度学习的实际标准；
- **训练循环四步**：前向 → 计算 Loss → 计算梯度 → 更新参数，这个闭环无论网络多深、框架多复杂，永远不变。

给零基础读者的一句总比喻

如果把训练神经网络比作学打靶：

- **损失函数** = 报靶员，告诉你偏了几环（MSE 对大偏差惩罚重，MAE 一视同仁，交叉熵对”完全没射中”惩罚最重）；
- **梯度** = 教练，告诉你”弓往左转、往右转还是往上抬”——梯度的方向就是需要调整的方向（取反）；
- **学习率** = 每次调整的幅度：调太猛（大学习率），可能从左偏变成右偏；调太保守（小学习率），射 100 箭还没对准；
- **Mini-batch SGD** = 不等你把靶场所有人的意见都收集完再调，每次随机问一小组人，马上调，虽然每次不完美，但总体射得越来越准——快！